Mathematical Foundations

Emission-Trajectory ML Project

Dr. Khawar Naeem
Qatar Transportation and Traffic Safety Center (QTTSC), Qatar University

13 July 2026 (living document; grows with each modeling session)

Abstract

This document formalizes the mathematics behind the emission-trajectory forecasting pipeline (`https://github.com/khawar89/energy-carbon-forecast-ml`): the evaluation metrics every model is scored by, the two baselines that define the bar a model must clear, Ridge regression as the project's first learned model, and the ensemble and boosting methods that follow it. Each equation is numbered and tied directly to the corresponding code in `src/evaluate.py`, `notebooks/03_baselines.ipynb`, and `notebooks/04_models_ridge_tree.ipynb`. New sections are added as later sessions (XGBoost, error analysis) are built, so this remains a growing companion to the coding sessions rather than a one-time summary. This is intended to support future methodology writing and research extension of the project, not to substitute for it.

Contents

1 Notation

This document uses consistent notation across all sections, matching the pipeline built in `src/build_features.py` and `src/evaluate.py`.

- c indexes a country; t indexes a calendar year.
- $y_{c,t}$ denotes the observed production-based CO_2 emissions (Mt) of country c in year t.
- The *feature year* t is the year whose information is known at prediction time; the *target year* $t+1$ is the year being predicted. No feature used to predict $y_{c,t+1}$ may use information from after year t (see the project's `CLAUDE.md` and `AGENTS.md` for the full leakage-prevention rationale; every equation in this document assumes that constraint is already enforced).
- $\hat{y}_{c,t+1}$ denotes a model's prediction of $y_{c,t+1}$, made using information available through year t.
- $x_{c,t} \in \mathbb{R}^p$ denotes the feature vector for country c at year t (lags, rolling statistics, population, and related columns), with p the number of features.
- n denotes the number of (country, year) rows in a given evaluation set (for example, the validation split).
- Splits are labeled train, val (validation), and test, assigned by the *target* year $t+1$, not the feature year t: train covers target years through 2014, val covers 2015–2018, test covers 2019–2023, and a separate provisional table covers target year 2024.

2 Evaluation Metrics

Every model in this project, from the persistence baseline through XGBoost, is scored by the same set of metrics, implemented once in `src/evaluate.py` and reused unchanged in every later session. This section states each metric formally.

2.1 Mean Absolute Error (MAE)

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n} |y_i - \hat{y}_i| \tag{1}$$

MAE is the average size of a miss, in the original units (Mt CO_2). It is interpretable, but because errors are measured in absolute units, its value is dominated by the largest countries by emissions level.

2.2 Root Mean Squared Error (RMSE)

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2} \tag{2}$$

Squaring each error before averaging penalizes large misses quadratically. Consequently RMSE $\gg$ MAE for a given model signals that its error budget is concentrated in a small number of large misses (a "fat tail") rather than spread evenly across rows.

2.3 Median Absolute Error (MedianAE)

$$\text{MedianAE} = \text{median}_i\,|y_i - \hat{y}_i| \tag{3}$$

MedianAE is immune to the largest emitters: it reports the typical error for the typical country. Comparing Equations 1 and 3 on the same model reveals how much of the average error is attributable to a handful of very large countries rather than to the rest of the sample, a scale-dependence issue discussed generally by [7].

2.4 Median Absolute Percentage Error (MdAPE)

$$\text{MdAPE} = 100 \times \text{median}_{i\,:\,y_i \geq \tau}\left(\frac{|y_i - \hat{y}_i|}{y_i}\right) \tag{4}$$

where τ is a minimum-level threshold (this project uses $\tau = 1$ Mt). Percentage errors are undefined or explosive as $y_i \to 0$; the exclusion in Equation 4 is part of the metric's definition, not an afterthought, following the general caution around percentage-based accuracy measures raised by [7].

2.5 Persistence Skill Score

$$\text{Skill} = 1 - \frac{\text{MAE}_{\text{model}}}{\text{MAE}_{\text{persistence}}} \tag{5}$$

The skill score in Equation 5 measures a model's error relative to the persistence baseline (Section 3), rather than in absolute terms. Skill = 0 means the model is exactly as accurate as assuming no change; Skill = 1 is the unreachable limit of zero error; negative skill means the model is less accurate than doing nothing and must be reported as a loss, not merely a smaller win.

2.6 The same-rows rule

Every comparison table in this project scores all candidate models on an identical set of rows: a row missing any model's prediction, or missing the target itself, is dropped for every model before Equations 1–5 are computed (`evaluate.comparison_table`). Without this rule, a model could appear to win only because it was scored on an easier subset of rows.

3 Baselines

A baseline is a model that requires no fitting: no parameters are estimated from data. Its purpose is to define the bar every learned model must clear (Section 2's skill score is measured against it).

3.1 Persistence

$$\hat{y}_{c,t+1} = y_{c,t} \tag{6}$$

Equation 6 predicts that next year's level equals this year's level. This is the naive forecast, and it is the optimal forecast under a random-walk-without-drift model of the underlying series [6]. It is a strong baseline for country-level CO_2 specifically because the underlying physical stock (power plants, vehicle fleets, industrial capital) changes slowly: the median absolute year-over-year change in this dataset since 2010 is approximately 4.4% of the level (Session 1 EDA).

3.2 Linear Trend (Drift)

$$\hat{y}_{c,t+1} = y_{c,t} + \bar{\Delta}^{(5)}_{c,t}, \qquad \bar{\Delta}^{(5)}_{c,t} = \frac{y_{c,t} - y_{c,t-5}}{5} \tag{7}$$

Equation 7 extrapolates the average yearly change over the preceding five years one step forward. It is this project's five-year-windowed variant of the classical drift method [6], which in its general form draws a line between a series' first and last observed points and extrapolates it. The trade-off is explicit: Equation 7 outperforms Equation 6 for a country on a sustained trajectory, and underperforms it the moment a country's direction reverses, since the linear extrapolation has no mechanism to anticipate a turn.

3.3 Bias direction

For any series with a genuine sustained trend, persistence (Equation 6) is a biased estimator of $y_{c,t+1}$: if $y_{c,t}$ is rising, persistence systematically under-predicts; if $y_{c,t}$ is falling (for example, a country whose emissions peaked and have since declined), persistence systematically over-predicts. The sign of the bias tracks the sign of the country's own recent trend, not a fixed direction.

4 Ridge Regression

4.1 Ordinary least squares

Given a design matrix $X \in \mathbb{R}^{n \times p}$ (rows are (country, year) observations, columns are features) and target vector $y \in \mathbb{R}^n$, ordinary least squares (OLS) solves

$$\hat{\beta}_{\text{OLS}} = \arg\min_{\beta} \|y - X\beta\|_2^2 \tag{8}$$

with closed-form solution $\hat{\beta}_{\text{OLS}} = (X^\top X)^{-1} X^\top y$, provided $X^\top X$ is invertible.

4.2 The problem with correlated features

This project's feature set includes multiple lags and rolling statistics of the same underlying series (`co2_lag1`, `co2_lag3`, `co2_lag5`, `co2_lag10`, `co2_roll5_mean`, `co2_roll10_mean`), which are, by construction, highly correlated with one another. When columns of X are strongly correlated, $X^\top X$ is close to singular, and $\hat{\beta}_{\text{OLS}}$ becomes highly sensitive to small changes in the data: its variance inflates even though it remains unbiased.

4.3 Ridge regression

Ridge regression adds an ℓ_2 penalty on the coefficients:

$$\hat{\beta}_{\text{ridge}} = \arg\min_{\beta} \|y - X\beta\|_2^2 + \alpha \|\beta\|_2^2, \qquad \alpha \geq 0 \tag{9}$$

with closed-form solution

$$\hat{\beta}_{\text{ridge}} = \left(X^\top X + \alpha I\right)^{-1} X^\top y \tag{10}$$

Adding αI to $X^\top X$ in Equation 10 guarantees invertibility for any $\alpha > 0$ and shrinks the coefficients of correlated features toward one another and toward zero. This trades a small amount of bias for a reduction in variance [5], and is the reason this project prefers Ridge over OLS given its correlated lag and rolling-statistic feature set.

4.4 Preprocessing

Because the ridge penalty in Equation 9 is applied uniformly to all coefficients, features must be on comparable scales before fitting; unlike tree-based models (Section 5), Ridge requires feature scaling, and that scaler must be fit on training rows only, to avoid leaking validation or test distributional information into its parameters.

5 Ensembles and Boosting

5.1 Regression trees

A regression tree partitions the feature space into disjoint regions $R_1, \ldots, R_J$ and predicts the mean target value within each region. A single split at feature j, threshold s is chosen to minimize the total sum of squared residuals across the two resulting regions:

$$(j^*, s^*) = \arg\min_{j,s} \left[\sum_{i:\, x_i \in R_1(j,s)} (y_i - \bar{y}_{R_1})^2 + \sum_{i:\, x_i \in R_2(j,s)} (y_i - \bar{y}_{R_2})^2 \right] \tag{11}$$

Trees require no feature scaling: Equation 11 depends only on the ordering of feature values within a column, not their magnitude, unlike the Ridge objective in Equation 9.

5.2 Bagging

Bagging (bootstrap aggregating) reduces variance by averaging many trees, each fit to an independent bootstrap resample of the training rows:

$$\hat{f}_{\text{bag}}(x) = \frac{1}{B} \sum_{b=1}^{B} \hat{f}_b(x) \tag{12}$$

where each $\hat{f}_b$ is a tree fit to bootstrap sample b [1]. Random forests extend Equation 12 by additionally restricting each split in Equation 11 to a random subset of features, further decorrelating the individual trees [2].

5.3 Gradient boosting

Where bagging averages independent trees fit in parallel, gradient boosting fits trees sequentially, each one correcting the errors of the ensemble built so far. The model is an additive expansion

$$F_M(x) = \sum_{m=1}^{M} \gamma_m h_m(x) \tag{13}$$

built greedily: at each stage m, tree h_m is fit to the negative gradient (pseudo-residual) of the loss function L with respect to the current model's predictions,

$$r_{i,m} = - \left. \frac{\partial L(y_i, F(x_i))}{\partial F(x_i)} \right|_{F=F_{m-1}} \tag{14}$$

which for squared-error loss reduces to the ordinary residual $y_i - F_{m-1}(x_i)$. This is gradient descent carried out in the space of functions rather than parameters [4]. `HistGradientBoostingRegressor` (scikit-learn), used in this project's Session 4, implements Equations 13–14 with histogram-binned feature values, which reduces the split-search cost of Equation 11 from scanning every unique value to scanning a fixed number of bins, and handles missing feature values natively.

5.4 XGBoost

XGBoost augments gradient boosting with an explicit regularization term and a second-order (Newton) approximation of the loss. Its objective at boosting round m is

$$\mathcal{L}^{(m)} = \sum_{i=1}^{n} l(y_i,\, F_{m-1}(x_i) + h_m(x_i)) + \Omega(h_m), \qquad \Omega(h) = \gamma T + \tfrac{1}{2}\lambda \|w\|^2 \tag{15}$$

where T is the number of leaves in tree h_m and w its leaf weights. Rather than fitting h_m to the first-order pseudo-residual alone (Equation 14), XGBoost takes a second-order Taylor expansion of the loss around F_{m-1}, using the per-observation gradient g_i and Hessian h_i of l, which yields a closed-form optimal leaf weight and an explicit split-gain criterion balancing improvement against the Ω penalty in Equation 15 [3]. This project treats XGBoost as a comparison model, not the project's goal (see `AGENTS.md`), with a small, defensible hyperparameter grid selected on the validation years rather than a wide search. This section will be extended with the full second-order derivation and the exact split-gain formula when Session 5 (XGBoost) is built.

References

[1] Leo Breiman. Bagging predictors. *Machine Learning*, 24(2):123–140, 1996.

[2] Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001.

[3] Tianqi Chen and Carlos Guestrin. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, pages 785–794, 2016. Free preprint: arXiv:1603.02754.

[4] Jerome H. Friedman. Greedy function approximation: A gradient boosting machine. *The Annals of Statistics*, 29(5):1189–1232, 2001.

[5] Arthur E. Hoerl and Robert W. Kennard. Ridge regression: Biased estimation for nonorthogonal problems. *Technometrics*, 12(1):55–67, 1970.

[6] Rob J. Hyndman and George Athanasopoulos. *Forecasting: Principles and Practice*. OTexts, Melbourne, Australia, 3rd edition, 2021. Free online textbook.

[7] Rob J. Hyndman and Anne B. Koehler. Another look at measures of forecast accuracy. *International Journal of Forecasting*, 22(4):679–688, 2006.